GPIO Driver Analysis

Prepared By: Anagha

Introduction

GPIO is the primary interface between the microcontroller and external hardware. In this project, a custom GPIO driver was developed for STM32F446RE to abstract direct register access and provide reusable APIs. The driver allows configuration of pin modes, speed, pull-up/pull-down settings, output type, and alternate functions. GPIO is the primary interface between the microcontroller and external hardware. In this project, a custom GPIO driver was developed for STM32F446RE to abstract direct register access and provide reusable APIs. The driver allows configuration of pin modes, speed, pull-up/pull-down settings, output type, and alternate functions.

Driver Architecture

The application interacts with GPIO APIs instead of directly modifying registers. The GPIO driver sits between the application and hardware registers, improving readability, modularity, and portability. The application interacts with GPIO APIs instead of directly modifying registers. The GPIO driver sits between the application and hardware registers, improving readability, modularity, and portability.

Data Structures

The driver uses gpio_pin_config_t to store pin configuration parameters and gpio_handle_t to combine the port address and configuration. This handle-based design makes the driver reusable across multiple GPIO ports. The driver uses gpio_pin_config_t to store pin configuration parameters and gpio_handle_t to combine the port address and configuration. This handle-based design makes the driver reusable across multiple GPIO ports.

Component	Purpose
Configuration Structure	Stores settings
Handle Structure	Stores peripheral and config
APIs	Interface to application

Register Analysis

MODER configures pin modes, OTYPER controls output type, OSPEEDR controls speed, PUPDR configures pull resistors, IDR reads input values, ODR writes outputs, BSRR performs atomic set/reset operations, and AFR selects alternate functions. MODER configures pin modes, OTYPER controls output type, OSPEEDR controls speed, PUPDR configures pull resistors, IDR reads input values, ODR writes outputs, BSRR performs atomic set/reset operations, and AFR selects alternate functions.

API Analysis

gpio_clock_control() enables clocks. gpio_init() configures registers. gpio_read_pin() reads pin state. gpio_write_pin() controls outputs. gpio_toggle_pin() changes pin state. Additional APIs support pull-up configuration, speed selection, and direction control. gpio_clock_control() enables clocks. gpio_init() configures registers. gpio_read_pin() reads pin state. gpio_write_pin() controls outputs. gpio_toggle_pin() changes pin state. Additional APIs support pull-up configuration, speed selection, and direction control.

Application Example

The driver was validated using a push button connected to PC13 and an LED connected to PA5. Pressing the button toggles the LED. Another example uses PA6 and PA7 for alternate LED blinking. The driver was validated using a push button connected to PC13 and an LED connected to PA5. Pressing the button toggles the LED. Another example uses PA6 and PA7 for alternate LED blinking.

Advantages and Conclusion

The GPIO driver provides hardware abstraction, reusable APIs, and simplified development. The implementation demonstrates understanding of STM32 register-level programming. The GPIO driver provides hardware abstraction, reusable APIs, and simplified development. The implementation demonstrates understanding of STM32 register-level programming.